

Universidade Federal de Pernambuco

Centro de Informática – CIn

Graduação em Engenharia da Computação

Introdução à Aprendizagem Profunda

Atividade Prática Supervisionada

Implementação e Comparação de Redes Convolucionais
(CNN)

Disciplina: Introdução à Aprendizagem Profunda

Datasets avaliados: MNIST e CIFAR-10

Framework: PyTorch

Modelo de comparação: ResNet-50 (pré-treinada no ImageNet)

Recife, 13 de maio de 2026

0 Sumário

1	Introdução	2
2	Metodologia	2
2.1	Configuração Geral do Ambiente	2
2.2	Pipelines de Dados	2
2.3	Arquitetura CNN – MNIST	2
2.4	Arquitetura CNN – CIFAR-10	3
2.5	Baseline: ResNet-50 com Transfer Learning	4
3	Resultados	4
3.1	Acurácias Finais	4
3.2	Curvas de Treinamento	4
3.3	Comparação Final entre Modelos	6
4	Discussão	6
4.1	MNIST: Tarefa Simples, CNN Rasa é Suficiente	6
4.2	CIFAR-10: A Distância entre Complexidade e Capacidade	7
5	Propostas de Melhoria	7
6	Desafio: Visualização de Filtros e Ativações	8
6.1	Motivação	8
6.2	Pesos dos Filtros (Kernels)	8
6.3	Mapas de Ativação	9
6.4	Interpretação dos Resultados Visuais	10
7	Conclusão	10

1 Introdução

Este relatório documenta a implementação, o treinamento e a análise comparativa de CNNs desenvolvidas em PyTorch para os conjuntos de dados MNIST e CIFAR-10, confrontando seus resultados com os obtidos por uma rede pré-treinada de alta capacidade, a ResNet-50.

O trabalho está estruturado em cinco eixos principais: (i) descrição das escolhas metodológicas de arquitetura e regularização; (ii) apresentação quantitativa e visual dos resultados de treinamento; (iii) discussão analítica sobre as diferenças de desempenho observadas; (iv) propostas concretas de melhoria; e (v) a exploração do desafio proposto de visualização das ativações e pesos dos filtros convolucionais.

2 Metodologia

2.1 Configuração Geral do Ambiente

Todos os experimentos foram conduzidos no Google Colab, com aceleração via GPU CUDA. Os hiperparâmetros globais adotados foram: `BATCH_SIZE = 64`, `EPOCHS = 15`, `LR = 1e-3` (otimizador Adam) e `weight_decay = 1e-4` (regularização L2). O scheduler `StepLR` reduziu a taxa de aprendizado à metade a cada 5 épocas, refinando a convergência nas épocas finais, limitadas pelo mecanismo de *Early Stopping* com `patience = 3` e `min_delta = 1e-4`.

2.2 Pipelines de Dados

Os dados foram carregados via `torchvision.datasets` com transformações distintas por dataset:

- **MNIST:** normalização com média $\mu = 0,1307$ e desvio padrão $\sigma = 0,3081$, valores específicos do dataset.
- **CIFAR-10 (treino):** `RandomHorizontalFlip` e `RandomCrop(32, padding=4)`, seguidos de normalização por canal RGB $(\mu_R, \mu_G, \mu_B) = (0,4914, 0,4822, 0,4465)$ e $(\sigma_R, \sigma_G, \sigma_B) = (0,2023, 0,1994, 0,2010)$. O *flip* horizontal cria invariância a espelhamentos, enquanto o *crop* com preenchimento simula translações, ampliando efetivamente a diversidade do dataset sem custo de coleta.

2.3 Arquitetura CNN – MNIST

Para o MNIST, adotou-se uma arquitetura deliberadamente rasa, composta por dois blocos convolucionais seguidos de camadas densas. O fluxo de processamento é:

$$\text{Conv}_{32} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Drop}_{0,25} \rightarrow \text{Conv}_{64} \rightarrow \text{ReLU} \rightarrow \\ \rightarrow \text{MaxPool} \rightarrow \text{Drop}_{0,25} \rightarrow \text{FC}_{128} \rightarrow \text{Drop}_{0,5} \rightarrow \text{FC}_{10}$$

Justificativa das escolhas de regularização:

- **Weight Decay (L2, $\lambda = 10^{-4}$):** adiciona ao gradiente um termo proporcional ao peso, $\nabla \mathcal{L} += \lambda w$, penalizando soluções de alta norma. O valor conservador 10^{-4} evita sub-aprendizado em uma tarefa de baixa complexidade como o MNIST.
- **Dropout leve ($p = 0,25$ nas convoluções / $p = 0,5$ na FC):** desativa aleatoriamente neurônios durante o treino, forçando redundância representacional. Nas camadas convolucionais, um valor baixo preserva os mapas de features espaciais, que já são compactos (7×7 ao final do segundo bloco).

2.4 Arquitetura CNN – CIFAR-10

O CIFAR-10 demanda uma rede mais profunda, capaz de aprender uma hierarquia semântica progressiva: bordas $\rightarrow$ texturas $\rightarrow$ partes $\rightarrow$ objetos. Adotou-se uma arquitetura de três blocos convolucionais com filtros crescentes ($32 \rightarrow 64 \rightarrow 128$):

$$[\text{Conv}_C \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Drop}_{0,3}]_{\times 3} \rightarrow \text{FC}_{256} \rightarrow \text{Drop}_{0,5} \rightarrow \text{FC}_{10}$$

Justificativa das escolhas de projeto:

- **Batch Normalization (BN):** normaliza as ativações de cada mini-batch, reduzindo o *internal covariate shift*. Além de estabilizar e acelerar o treinamento, atua como regularizador implícito ao introduzir ruído nas estimativas das estatísticas de batch.
- **Dropout moderado ($p = 0,3$ / $0,5$):** a maior variância intra-classe do CIFAR-10 e seu tamanho limitado (50k imagens de treino) tornam o overfitting um risco concreto. O valor 0,3 nas convoluções equilibra capacidade e generalização, enquanto 0,5 na FC suprime co-adaptações nas representações de alto nível.
- **Filtros crescentes ($32 \rightarrow 64 \rightarrow 128$):** canais progressivamente maiores permitem que camadas mais profundas combinem features de baixo nível em representações de maior dimensão e complexidade semântica.
- **Data Augmentation:** cria invariâncias geométricas durante o treino, reduzindo a dependência a posições e orientações específicas do objeto.

2.5 Baseline: ResNet-50 com Transfer Learning

Como modelo de comparação, utilizou-se a ResNet-50 pré-treinada no ImageNet, disponível via `torchvision.models`. As adaptações realizadas foram:

1. **Adaptação para escala de cinza (MNIST):** substituição da primeira camada convolucional (`conv1`) por uma equivalente de 1 canal de entrada.
2. **Substituição do classificador:** a camada `fc` original (1 000 classes do ImageNet) foi trocada por uma camada linear de 10 saídas.
3. **Redimensionamento das entradas:** todas as imagens foram redimensionadas para 224×224 pixels, conforme exigido pela arquitetura da ResNet-50.

O backbone convolucional foi mantido com seus pesos pré-treinados, e apenas o classificador final foi ajustado em 5 épocas de fine-tuning.

3 Resultados

3.1 Acurácias Finais

A Tabela 1 consolida as melhores acurácias obtidas no conjunto de teste de cada experimento.

Tabela 1: Comparação de acurácia no conjunto de teste

Modelo	Dataset	Melhor Acurácia (Teste)
CNN Custom	MNIST	99,38%
CNN Custom	CIFAR-10	68,02%
ResNet-50	MNIST	99,07%
ResNet-50	CIFAR-10	86,76%

3.2 Curvas de Treinamento

As Figuras 1 a 4 apresentam as curvas de *loss* e acurácia ao longo das épocas para cada experimento.

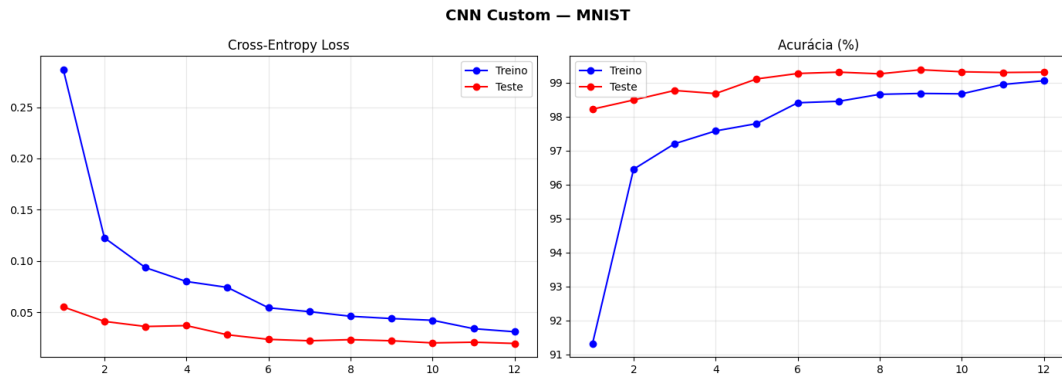


Figura 1: Curvas de $loss$ e acurácia – CNN Custom no MNIST.

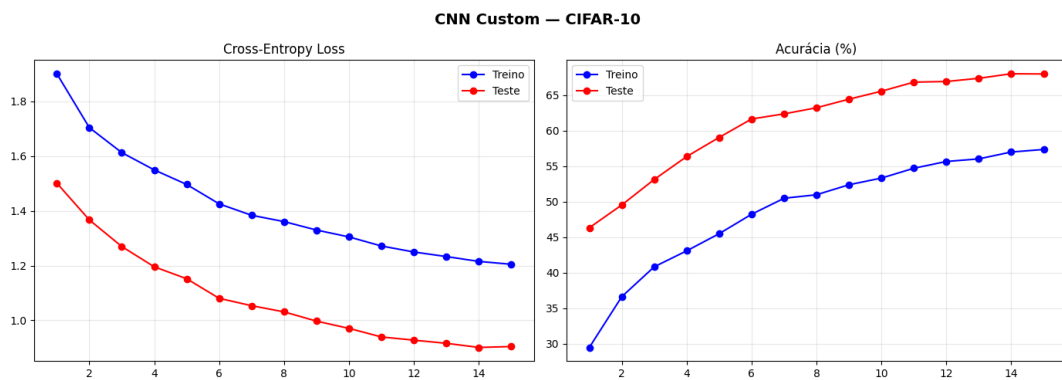


Figura 2: Curvas de $loss$ e acurácia – CNN Custom no CIFAR-10.

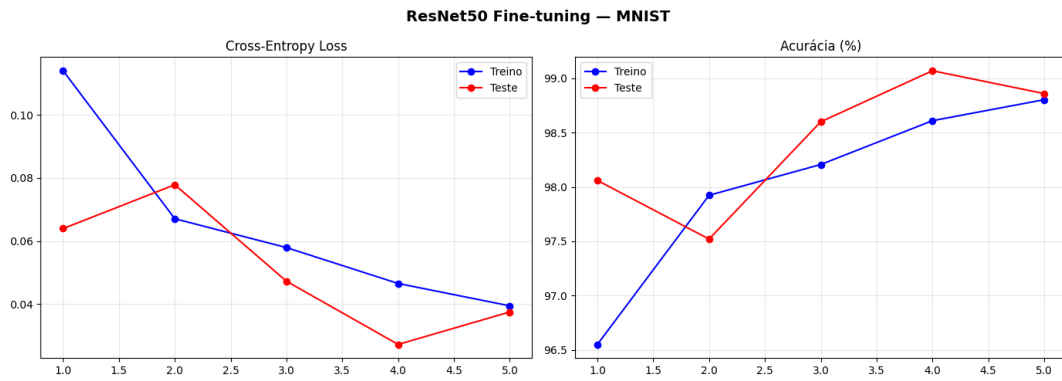


Figura 3: Curvas de $loss$ e acurácia – ResNet-50 no MNIST.

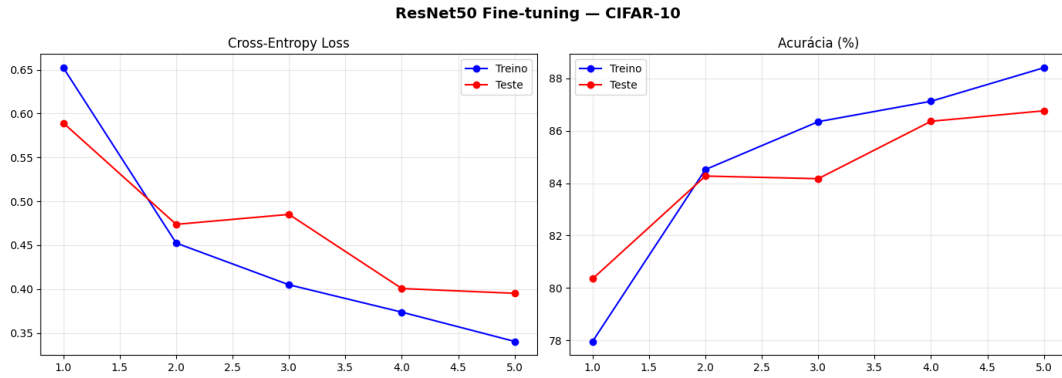


Figura 4: Curvas de *loss* e acurácia – ResNet-50 no CIFAR-10.

3.3 Comparação Final entre Modelos

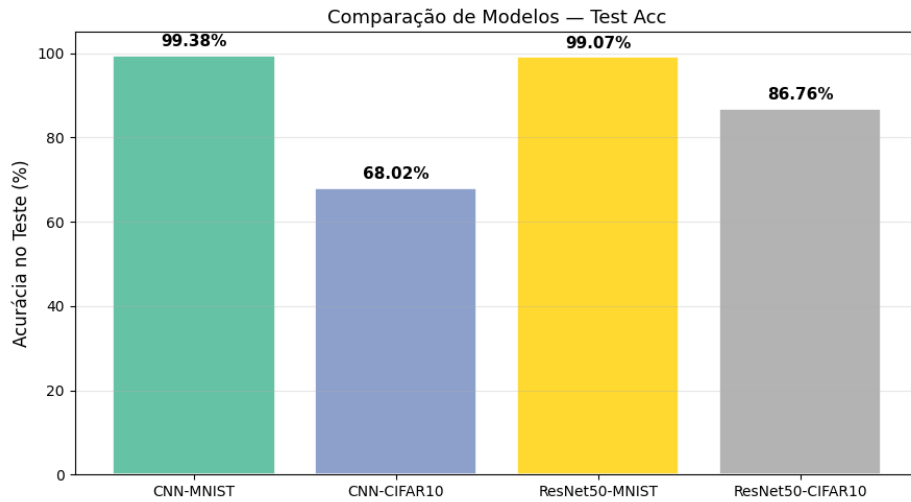


Figura 5: Comparação das melhores acurácias obtidas no conjunto de teste por cada configuração. A CNN Custom supera a ResNet-50 no MNIST, enquanto no CIFAR-10 a diferença de 18,74 pp em favor da ResNet-50 demonstra o impacto dos pesos pré-treinados em tarefas mais complexas.

4 Discussão

4.1 MNIST: Tarefa Simples, CNN Rasa é Suficiente

O MNIST é um dataset de baixa complexidade: imagens binárias 28×28 , fundo uniforme, variância intra-classe reduzida. A CNN Custom atingiu **99,38%** de acurácia, superando a ResNet-50 em fine-tuning (99,07%). Este resultado contraintuitivo se explica pois a CNN rasa, projetada especificamente para a dimensionalidade do MNIST, explora suas features de forma mais direta e eficiente.

O mecanismo de *Early Stopping* mostrou-se fundamental: sem ele, o modelo continuaria treinando além do ponto ótimo sem ganhos reais de generalização, desperdiçando

recursos computacionais.

4.2 CIFAR-10: A Distância entre Complexidade e Capacidade

A acurácia de 68,02% da CNN Custom no CIFAR-10, frente aos 86,76% da ResNet-50, evidencia o impacto direto da **capacidade arquitetural** e dos **pesos pré-treinados** no ImageNet. O CIFAR-10 possui alta variância intra-classe, demandando representações hierárquicas profundas que uma rede de 3 blocos convolucionais treinada do zero em 50k imagens não consegue capturar plenamente em apenas 15 épocas.

A ResNet-50, com seus 50 blocos residuais e pesos que já codificam detectores de texturas, formas e partes de objetos aprendidos em 1,2 milhões de imagens, parte de uma representação de features muito mais rica. O fine-tuning apenas redireciona essas features para as 10 classes do CIFAR-10, o que explica a convergência rápida e o alto desempenho mesmo com apenas 5 épocas de ajuste.

Outro ponto relevante é a curva de aprendizado da CNN Custom no CIFAR-10: a acurácia de teste ainda estava crescendo monotonicamente ao final da 15ª época, sugerindo que a rede não havia saturado seu potencial. Com mais épocas e hiperparâmetros ajustados, ganhos adicionais seriam esperados.

5 Propostas de Melhoria

A acurácia de 68,02% da CNN Custom no CIFAR-10 deixa ampla margem para aprimoramentos. As estratégias a seguir, baseadas em evidências da literatura, representam os caminhos mais promissores:

1. **Aumento de profundidade e uso de conexões residuais:** implementar blocos residuais (*skip connections*) permite treinar redes muito mais profundas sem degradação por *vanishing gradient*. Uma arquitetura estilo ResNet com 10–20 camadas provavelmente elevaria a acurácia para a faixa de 85–90%.
2. **Augmentation mais agressiva:** técnicas como ColorJitter, RandomRotation, CutOut e MixUp aumentam a robustez da rede a variações de cor, orientação e oclusão, problemas frequentes no CIFAR-10.
3. **Ajuste do agendamento de taxa de aprendizado:** substituir o StepLR por um CosineAnnealingLR proporciona decaimento suave até zero, explorando melhor os mínimos rasos da função de perda e geralmente resultando em 1–3 pp de ganho.
4. **Aumento do número de épocas com Early Stopping:** dado que a curva de aprendizado ainda estava ascendente aos 15 épocas, treinar por até 50–100

épocas com o Early Stopping como salvaguarda provavelmente traria ganhos significativos.

5. **Transfer Learning parcial:** utilizar um backbone pré-treinado (e.g., EfficientNet-B0 ou MobileNetV3) com fine-tuning seletivo nas últimas camadas representa o caminho mais eficiente para superar 90% com recursos computacionais limitados.
6. **Label Smoothing:** substituir os rótulos *one-hot* por distribuições suavizadas (e.g., $\epsilon = 0,1$) reduz a confiança excessiva do modelo e melhora a calibração, resultando em melhor generalização.

6 Desafio: Visualização de Filtros e Ativações

6.1 Motivação

Redes convolucionais frequentemente são tratadas como “caixas pretas”. A visualização dos pesos dos filtros e dos mapas de ativação da primeira camada convolucional oferece uma janela interpretável para o que a rede está aprendendo, permitindo depurar comportamentos inesperados e validar se as representações aprendidas são coerentes com o domínio do problema.

6.2 Pesos dos Filtros (Kernels)

Os pesos brutos da primeira camada convolucional codificam os detectores de features de mais baixo nível que a rede aplicará sobre a imagem de entrada. Após o treinamento, é comum observar filtros que se especializam em detectar bordas horizontais, verticais, diagonais, gradientes de cor e texturas simples, comportamento análogo ao das células simples do córtex visual primário (V1) descritas por Hubel e Wiesel (1962). A normalização dos pesos para $[0, 1]$ antes da visualização facilita a interpretação visual.

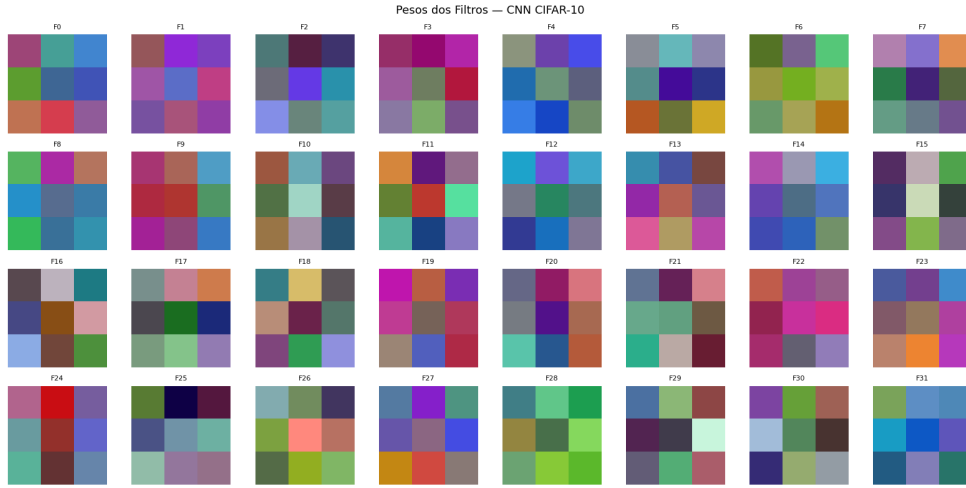


Figura 6: Pesos (kernels) da primeira camada convolucional após o treinamento. Cada miniatura representa um filtro 3×3 . Padrões de alto contraste correspondem a detectores de borda e gradiente; padrões difusos indicam filtros de suavização (passa-baixa).

6.3 Mapas de Ativação

Os mapas de ativação mostram *quais regiões* de uma imagem específica ativaram cada filtro da primeira camada. Filtros detectores de borda produzirão ativações fortes nas regiões de transição de luminância; filtros de textura responderão a padrões repetitivos. Regiões escuras indicam ativação nula (ReLU), enquanto regiões brilhantes indicam alta resposta ao padrão codificado pelo filtro.

A implementação utilizou *hooks* de *forward pass* via `register_forward_hook`, interceptando a saída da camada alvo sem necessidade de modificar a arquitetura da rede.

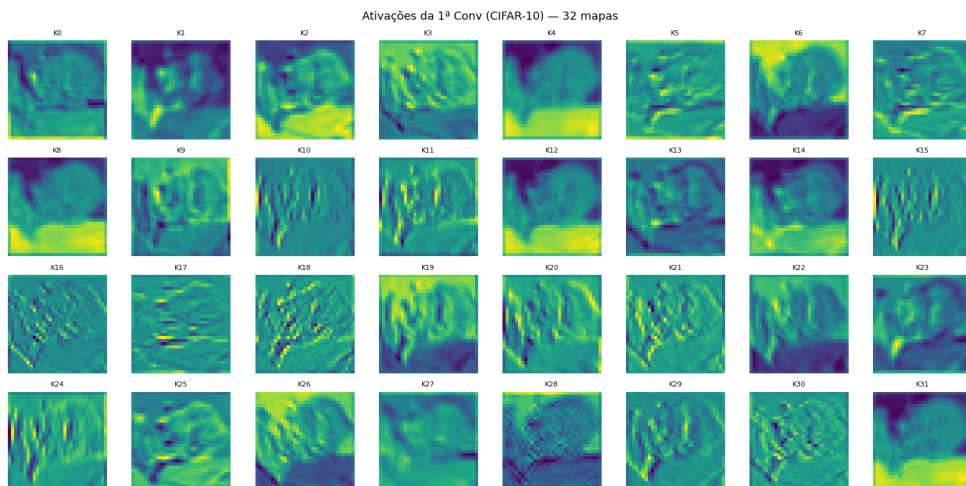


Figura 7: Mapas de ativação da primeira camada convolucional para uma imagem de entrada representativa. Cada mapa corresponde a um dos 32 filtros da camada. A diversidade dos padrões ativados confirma a especialização dos filtros em diferentes features de baixo nível.

6.4 Interpretação dos Resultados Visuais

A comparação entre os filtros da CNN treinada no MNIST e no CIFAR-10 revela uma diferença qualitativa relevante: no MNIST, os filtros em escala de cinza tendem a capturar estruturas de borda simples e uniformes, dada a natureza binária das imagens de dígitos. No CIFAR-10, os filtros RGB apresentam maior riqueza cromática e padrões mais complexos, refletindo a maior diversidade de texturas e cores presentes nas imagens naturais. Esta diferença qualitativa é coerente com a diferença de complexidade entre os dois datasets.

7 Conclusão

Este trabalho demonstrou que a escolha arquitetural deve ser proporcional à complexidade da tarefa. Para o MNIST, uma CNN rasa com regularização adequada (Dropout + Weight Decay) é suficiente para atingir acurácias superiores a 99%, inclusive superando um modelo de alta capacidade como a ResNet-50 em fine-tuning.

Para o CIFAR-10, a lacuna de 18,74 pp entre a CNN Custom (68,02%) e a ResNet-50 (86,76%) ilustra de forma contundente o valor do transfer learning: pesos pré-treinados em larga escala fornecem representações de features universais que redes treinadas do zero em datasets pequenos dificilmente replicam com recursos computacionais e épocas limitadas.

As técnicas de regularização aplicadas (Batch Normalization, Dropout, Data Augmentation e Early Stopping) foram eficazes em prevenir o overfitting, como evidenciado pela proximidade entre as curvas de treino e teste. As propostas de melhoria apontam caminhos concretos para elevar o desempenho da CNN Custom no CIFAR-10 para a faixa de 85–90% sem recorrer a arquiteturas pré-treinadas.